

Communication

Date	14 July 2026
Team ID	
Project Name	Personalized Networking Assistant
Maximum Marks	1 Mark

Communication Plan:

This document defines the communication approach used to manage the development, testing, deployment, and documentation of the Personalized Networking Assistant. As an individually maintained project, communication focuses on mentor reviews, GitHub-based progress tracking, issue resolution, deployment verification, and final submission preparation.

S.No	Communication Type	Frequency	Channel / Tool	Participants	Purpose
1	Development Progress Review	Regularly	GitHub / Local Development	Project Developer	Track implementation of frontend, backend, database, AI generation, and factual verification.
2	Mentor / Project Review	As Required	SkillWallet / Direct Review	Developer and Mentor / Reviewer	Review project progress, required deliverables, documentation, and necessary corrections.
3	Issue and Bug Resolution	As Needed	GitHub / Development Tools	Project Developer	Resolve React, TypeScript, Express, SQLite, Gemini, Wikipedia, build, and deployment issues.
4	Deployment Verification	After Major Updates	Render / GitHub	Project Developer	Verify production build, environment configuration, API behavior, and persistent database setup.
5	Documentation Review	Before Submission	GitHub / PDF Review	Developer and Reviewer	Check phase documents, diagrams, screenshots, project documentation, and formatting accuracy.
6	Final Demonstration Preparation	Once Before Submission	Deployed Application / Presentation	Project Developer	Perform a complete walkthrough of registration, profile setup, factual lookup, Gemini generation, history, feedback, and analytics.

Communication Challenges & Resolutions

S.No	Challenge Faced	Resolution / Action Taken
1	Coordinating the React frontend with the Node.js and Express backend while keeping shared TypeScript structures consistent.	Used a modular full-stack structure with reusable frontend components, shared types, dedicated API routes, and separate backend services.
2	Integrating Google Gemini generation and Wikipedia factual verification without exposing sensitive API credentials in the browser.	Moved external integrations to server-side services, loaded secrets from environment variables, and returned controlled responses through backend endpoints.
3	Maintaining SQLite data reliably during production deployment.	Configured the application to support a persistent database path and documented Render persistent-disk setup using DATABASE_URL.
4	Keeping project documentation aligned with the actual implementation and removing outdated sample-project technologies.	Used the current repository implementation and matching Markdown documentation as the source of truth for technical content and final PDF updates.